ALPE FORMACIÓN · SANTANDER

Edición 2026 · PHP 8.x

CURSO DE PHP · CLASE 12

Formularios, archivos y ficheros

- Qué es un formulario y qué pasa al pulsar «Enviar»
- El formulario más sencillo: dos páginas con GET
- El mismo formulario con POST
- ¿GET o POST? Cuándo usar cada uno
- Ver qué llega: var_dump($_GET) y print_r($_POST)
- Los tipos de campos y cómo llega cada uno
- Los datos que no llegan y los que llegan «sucios»
- ¿Se ha enviado el formulario? Tres formas de saberlo
- Un caso real con dos páginas: inscribirse en un taller
- Todo en la misma página
- ¿Una página o dos? Lo que gana y lo que pierde cada una
- Separar la lógica de la vista: la semilla del MVC
- El patrón del formulario profesional
- Validar: comprobar que el dato es correcto
- Escapar: htmlspecialchars y los ataques XSS
- Guardar datos en un fichero JSON
- Post/Redirect/Get: adiós al «¿Desea reenviar el formulario?»
- Subir archivos
- Para saber más: ficheros CSV



Ramón Abramo

+25 años de experiencia docente · Santander, Cantabria

ALPE

FORMACIÓN PROFESIONAL

🧭 Contenido de la clase

Hasta ahora éramos nosotros quienes escribíamos los datos en el código. Pero una web de verdad es **interactiva**: los usuarios escriben su nombre, eligen opciones, suben fotos y pulsan botones. Hoy aprendemos a **recibir** todo eso, a **comprobarlo** y a **guardarlo** en ficheros.

En la clase 6 hicimos una primera toma de contacto con los formularios. Hoy los vamos a ver **desde el principio y con calma**, como si fuera la primera vez: qué pasa al pulsar «Enviar», GET y POST, cada tipo de campo, cómo saber si el formulario se ha enviado... Y, cuando dominéis lo básico, construiremos un formulario «profesional»: que muestre los errores junto a cada campo, que no borre lo que el usuario ya había escrito y que no se deje colar código malicioso.

Os adelanto la frase de la clase, la que os voy a repetir hasta que la soñéis: **nunca os fiéis de lo que envía el usuario**.

1

📘 Qué es un formulario y qué pasa al pulsar «Enviar»



2

👍 El formulario más sencillo: dos páginas con GET



3

🚀 El mismo formulario con POST



4

🤔 ¿GET o POST? Cuándo usar cada uno



5

🔍 Ver qué llega: var_dump($_GET) y print_r($_POST)



6

👍 Los tipos de campos y cómo llega cada uno



7

⚠️ Los datos que no llegan y los que llegan «sucios»



8

😮 ¿Se ha enviado el formulario? Tres formas de saberlo



9

👍 Un caso real con dos páginas: inscribirse en un taller



10

🚀 Todo en la misma página



11

🤔 ¿Una página o dos? Lo que gana y lo que pierde cada una



12

😮 Separar la lógica de la vista: la semilla del MVC



13
👌 El patrón del formulario profesional



14
🤔 Validar: comprobar que el dato es correcto



15
🐞 Escapar: htmlspecialchars y los ataques XSS



16
👍 Guardar datos en un fichero JSON



17
😮 Post/Redirect/Get: adiós al «¿Desea reenviar el formulario?»



18
🚀 Subir archivos



19
🔍 Para saber más: ficheros CSV



20
🚀 Ejercicios



21
👋 Lo que tenéis que llevaros de hoy





CLASE 12 · PUNTO 1

📘 Qué es un formulario y qué pasa al pulsar «Enviar»

Un **formulario** es la forma que tiene una página web de **preguntar** algo al usuario: su nombre, una búsqueda, una contraseña, una foto... Ya visteis uno en la clase 6, pero hoy vamos a empezar desde cero y a entender bien cada pieza, porque es uno de los temas más importantes del curso.

Lo primero es tener claro que el formulario lo pinta el **navegador**, pero los datos los recibe **PHP**, en el servidor. Entre los dos hay un viaje:

Esquema

```
NAVEGADOR                                  SERVIDOR (PHP)
┌──────────────────────┐   1. petición con los   ┌──────────────────────┐
│ Nombre: [ Ana      ] │ ─── datos del form ──►  │ $_GET o $_POST       │
│        [ Enviar ]    │                         │   ["nombre" ⇒ "Ana"] │
└──────────────────────┘                         │ 2. PHP hace algo con ellos │
          ▲                                      └──────────┬───────────┘
          └──────────── 3. respuesta: una página HTML nueva ─┘
```

1. Al pulsar **Enviar**, el navegador recoge lo que hay escrito en cada campo y hace una **petición** a la página indicada.
2. PHP recibe los datos en unos arrays especiales, las **superglobales** `$_GET` y `$_POST`, disponibles en cualquier parte del código [1].
3. PHP genera una **respuesta**, una página HTML, y el navegador la muestra.

¿Recordáis el restaurante de la clase 1? El formulario es la **comanda**: el cliente (el usuario) la rellena, el camarero (la petición) la lleva a la cocina (PHP) y vuelve con el plato (la página de respuesta).

Las piezas de un formulario que **tenéis** que conocer [8]:

Pieza	Para qué sirve
`<form action="...">`	**Adónde** se envían los datos: la página PHP que los recibirá. Si no se pone, a la misma página
`<form method="...">`	**Cómo** viajan: `get` (en la URL) o `post` (dentro de la petición). Si no se pone, `get`
`name="..."` en cada campo	La **clave** con la que PHP leerá el dato. ¡Sin `name`, el campo no se envía!
`<button type="submit">`	El botón que envía el formulario
`<label for="...">` + `id="..."`	El texto de cada campo. No es obligatorio, pero ayuda (y es accesible): al pulsar el texto, se activa la caja



⚠️ **El error más frecuente:** escribir `name="Nombre"` en el formulario y leer `$_POST["nombre"]` en PHP. Mayúsculas y minúsculas cuentan: tienen que coincidir **exactamente**.



CLASE 12 · PUNTO 2

👍 El formulario más sencillo: dos páginas con GET

Empezamos por lo más básico: **dos archivos**. Uno **pregunta** (el formulario, HTML puro) y otro **responde** (PHP, que recibe los datos).

01-saludo-get.html — php-paso-a-paso/clase12 — Visual Studio Code

🌐 01-saludo-get.html

```html
<!-- 01-saludo-get.html: la página que PREGUNTA -->
<form action="02-saludo-get.php" method="get">
    <p>
        <label for="nombre">Nombre</label>
        <input type="text" id="nombre" name="nombre">
    </p>
    <p>
        <label for="edad">Edad</label>
        <input type="number" id="edad" name="edad">
    </p>
    <button type="submit">Saludar</button>
</form>
```

01-saludo-get.html — php-paso-a-paso/clase12 — Visual Studio Code

🐘 01-saludo-get.html

```php
<?php
declare(strict_types=1);

// 02-saludo-get.php: la página que RESPONDE
$nombre = $_GET["nombre"];
$edad = $_GET["edad"];
?>
<h1>¡Hola, <?php echo $nombre; ?>!</h1>
<p>Tienes <?php echo $edad; ?> años.</p>
```

📄 Ejemplos: `01-saludo-get.html` y `02-saludo-get.php`

Rellenadlo, pulsad **Saludar** y mirad con atención la barra de direcciones:

Esquema

```
http://php-paso-a-paso.test/clase12/02-saludo-get.php?nombre=Ana&edad=25
                                                      └──────── la «query string» ────────┘
```

- Detrás del `?` van los datos, en parejas `clave=valor` separadas por `&`.
- Cada `name` del formulario se ha convertido en una clave de `$_GET`: `$_GET["nombre"]` vale `"Ana"` y `$_GET["edad"]`, `"25"`.
- Fijaos en las comillas de `"25"`: **todo lo que llega de un formulario es texto**, aunque el campo sea `type="number"`. Si queréis operar con él, convertidlo: `(int) $_GET["edad"]`.

Ahora jugad con la URL: cambiad `Ana` por vuestro nombre y pulsad Intro. ¡El saludo cambia sin tocar el formulario! Con GET, los datos se pueden **escribir a mano, guardar en favoritos y compartir**.

GET sin formulario: los enlaces

Como los datos van en la URL, un simple **enlace** también envía datos por GET. Es lo que hacen las tiendas en línea con los filtros y las páginas: `productos.php?categoria=zapatos&pagina=2`.

03-enlaces-get.php — php-paso-a-paso/clase12 — Visual Studio Code

03-enlaces-get.php

```php
<?php
declare(strict_types=1);

$ciudad = $_GET["ciudad"] ?? "";

// http_build_query() codifica los caracteres especiales (espacios, ñ, &...) por nosotros
$enlaceCoruna = "03-enlaces-get.php?" . http_build_query(["ciudad" => "A Coruña"]);
?>
<a href="03-enlaces-get.php?ciudad=Madrid">Madrid</a>
<a href="<?= $enlaceCoruna ?>">A Coruña</a>
```

Mirad cómo queda el enlace de A Coruña: `?ciudad=A+Coru%C3%B1a`. En una URL no puede haber espacios ni eñes, así que se **codifican**: el espacio se convierte en `+` y la `ñ`, en `%C3%B1`. PHP los descodifica solo: en `$_GET["ciudad"]` vuelve a estar `"A Coruña"`.

📄 Ejemplo: `03-enlaces-get.php`



CLASE 12 · PUNTO 3

🚀 El mismo formulario con POST

Para usar POST cambian solo **dos cosas**: `method="post"` en el formulario y `$_POST` en lugar de `$_GET` en la página que recibe.

04-saludo-post.html — php-paso-a-paso/clase12 — Visual Studio Code

04-saludo-post.html

```html
<!-- 04-saludo-post.html -->
<form action="05-saludo-post.php" method="post">
    <p>
        <label for="usuario">Usuario</label>
        <input type="text" id="usuario" name="usuario">
    </p>
    <p>
        <label for="clave">Contraseña</label>
        <input type="password" id="clave" name="clave">
    </p>
    <button type="submit">Entrar</button>
</form>
```

04-saludo-post.html — php-paso-a-paso/clase12 — Visual Studio Code

04-saludo-post.html

```php
<?php
declare(strict_types=1);

// 05-saludo-post.php
$usuario = $_POST["usuario"];
$clave = $_POST["clave"];
?>
<h1>Bienvenido, <?php echo $usuario; ?></h1>
<p>Tu contraseña tiene <?php echo strlen($clave); ?> caracteres.</p>
```

📄 Ejemplos: `04-saludo-post.html` y `05-saludo-post.php`

Ahora la barra de direcciones dice solo `05-saludo-post.php`: los datos viajan **dentro** de la petición, en su **cuerpo**. ¿Os imagináis la contraseña escrita en la URL, en el historial del navegador y a la vista de cualquiera que mire la pantalla? Por eso un formulario de acceso **siempre** va por POST.



🔍 **¿Dónde están entonces los datos?** Abrid las herramientas del navegador (**F12**), pestaña **Red**, enviad el formulario y pulsad sobre la petición: en la sección **Carga** (*Payload*) aparecen `usuario` y `clave`. POST **oculta** los datos de la URL, pero **no los cifra**: eso lo hace HTTPS, el candado del navegador.

Otra diferencia que vais a notar: si recargáis (**F5**) una página que habéis recibido por POST, el navegador pregunta *«¿Desea reenviar el formulario?»*. Con GET no pregunta nada, porque los datos ya están en la URL. Veremos cómo evitar ese aviso en el punto 17.



CLASE 12 · PUNTO 4

🤔 ¿GET o POST? Cuándo usar cada uno

	GET	POST
Dónde viajan los datos	En la URL, tras el `?`: `buscar.php?q=zapatos`	Dentro del cuerpo de la petición
Se ven en la barra de direcciones y en el historial	✅ Sí	❌ No
Se pueden guardar en favoritos o compartir	✅ Sí	❌ No
Al recargar la página	Se repite sin preguntar	El navegador pregunta si reenviar
Cantidad de datos	Limitada (la URL no puede ser enorme)	Mucha, y también archivos
En PHP se leen en	`$_GET`	`$_POST`
Usadlo para...	**Consultar**: búsquedas, filtros, número de página	**Cambiar algo**: registros, logins, altas, bajas, pedidos, subir archivos

La regla para no equivocaros: **si el formulario solo consulta, GET; si cambia algo o lleva datos privados, POST**. Una búsqueda que se puede repetir mil veces sin consecuencias, GET. Un pedido que no queréis que se haga dos veces por recargar, POST.



⚠️ **Las contraseñas, siempre por POST.** Por GET acabarían en la barra de direcciones, en el historial del navegador y en los registros del servidor.



CLASE 12 · PUNTO 5

🔍 Ver qué llega: var_dump($_GET) y print_r($_POST)

Cuando un formulario «no funciona», casi siempre es porque el dato no llega con el nombre que esperáis. Antes de volveros locos, **mirad qué está llegando**:

06-ver-datos.php — php-paso-a-paso/clase12 — Visual Studio Code

06-ver-datos.php

```
1 <pre><?php var_dump($_GET); ?></pre>
2 <pre><?php print_r($_POST); ?></pre>
```

La etiqueta `<pre>` respeta los saltos de línea para que se lea bien. En este ejemplo hay dos formularios, uno por cada método. Probad los dos y observad cómo se llenan los arrays:

📄 Ejemplo: `06-ver-datos.php`

Fijaos en un detalle curioso: el formulario POST tiene `action="06-ver-datos.php?origen=formulario"`. Al enviarlo, llega `nombre` en `$_POST`... **y** `origen` en `$_GET`, porque está en la URL. Los dos arrays son independientes.

Para saber...	Usad
Con qué método se ha pedido la página	`$_SERVER["REQUEST_METHOD"]` → `"GET"` o `"POST"`
Qué ha llegado por la URL	`var_dump($_GET)`
Qué ha llegado en el cuerpo	`var_dump($_POST)`



💡 Existe también `$_REQUEST`, que mezcla los dos. **No lo uséis**: así nunca sabéis de dónde viene cada dato.



CLASE 12 · PUNTO 6

👍 Los tipos de campos y cómo llega cada uno

Un formulario tiene muchos más campos que la caja de texto. Cada uno llega a PHP de una forma [9], y algunos, si el usuario no los toca, **ni siquiera llegan**:

Campo	Qué llega a PHP	Si no se rellena o no se marca
`<input type="text">`, `email`, `password`, `<textarea>`	El texto escrito	Llega `""` (texto vacío)
`<input type="number">`	El número... **como texto**: `"25"`	Llega `""`
`<input type="date">`	Texto con el formato `AAAA-MM-DD`: `"2026-12-01"`	Llega `""`
`<input type="radio" name="turno" value="...">`	El `value` del marcado	❌ **No llega**
`<input type="checkbox" name="lenguajes[]" value="...">`	Un **array** con los `value` marcados (por el `[]`)	❌ **No llega**
Una casilla suelta, `<input type="checkbox" name="boletin">`	Su `value` (o `"on"` si no tiene)	❌ **No llega**
`<select name="ciudad">`	El `value` de la opción elegida	Siempre llega una
`<input type="hidden">`	Su `value`. El usuario no lo ve, pero se envía	Siempre llega
Un campo **sin** `name`	Nada	Nada

07-tipos-de-campos.html — php-paso-a-paso/clase12 — Visual Studio Code

07-tipos-de-campos.html

```html
<!-- Radio: todos con el MISMO name; solo se puede marcar uno -->
<label><input type="radio" name="turno" value="mañana"> Mañana</label>
<label><input type="radio" name="turno" value="tarde"> Tarde</label>

<!-- Casillas: mismo name terminado en []; llegan como un array -->
<label><input type="checkbox" name="lenguajes[]" value="HTML"> HTML</label>
<label><input type="checkbox" name="lenguajes[]" value="CSS"> CSS</label>

<select name="ciudad">
    <option value="">— Elige —</option>
    <option value="santander">Santander</option>
</select>
```

Y en PHP, cada uno con su red de seguridad:

07-tipos-de-campos.html — php-paso-a-paso/clase12 — Visual Studio Code

07-tipos-de-campos.html

```php
<?php
$nombre = trim($_POST["nombre"] ?? "");
$edad = (int) ($_POST["edad"] ?? 0);          // llega como TEXTO: lo convertimos
$turno = $_POST["turno"] ?? "sin elegir";      // radio sin marcar: no llega
$lenguajes = $_POST["lenguajes"] ?? [];        // casillas con []: un array (o nada)
$boletin = isset($_POST["boletin"]);           // casilla suelta: ¿existe o no?
```

📄 Ejemplos: `07-tipos-de-campos.html` y `08-recibir-campos.php`. La página que recibe muestra una tabla con cada dato y, debajo, el `var_dump($_POST)`: enviad el formulario sin marcar nada y comprobad qué claves faltan.



CLASE 12 · PUNTO 7

⚠️ Los datos que no llegan y los que llegan «sucios»

Los ejemplos de los puntos 2 y 3 funcionan... cuando todo va bien. Pero son muy confiados. Haced estas tres pruebas:

Prueba	Qué pasa
Abrid `02-saludo-get.php` directamente, sin pasar por el formulario	*Warning: Undefined array key "nombre"*: esa clave no existe en `$_GET`
Escribid `Ana`, con espacios delante y detrás	Los espacios llegan tal cual (y luego `"Ana" ≠ " Ana "`)
Escribid `<b>Ana</b>` como nombre	¡Sale en **negrita**! El navegador ha interpretado vuestro texto como HTML

La tercera es la más grave: si en lugar de `<b>` alguien escribe un `<script>`, el navegador lo **ejecuta**. Es un ataque llamado **XSS**, y lo veremos a fondo en el punto 15.

Estas son las herramientas que lo arreglan, y que a partir de ahora usaremos **siempre**:

Herramienta	Qué hace	Ejemplo
`??`	Si la clave no existe, usa un valor por defecto: adiós al *Warning*	`$_POST["nombre"] ?? ""`
`isset()`	¿Existe la clave? Devuelve `true` o `false`	`isset($_POST["boletin"])`
`trim()`	Quita los espacios del principio y del final	`trim(" Ana ")` → `"Ana"`
`(int)`, `(float)`	Convierte el texto en número	`(int) "25"` → `25`
`htmlspecialchars()`	Convierte `<`, `>`, `"`... en texto que se ve pero **no se ejecuta** [4]	`<b>` → `<b>`

La línea que vais a escribir cientos de veces es esta:

ejemplo.php — php-paso-a-paso/clase12 — Visual Studio Code

ejemplo.php

```php
$nombre = trim($_POST["nombre"] ?? "");   // existe o vale "", y sin espacios
```

Y al mostrarlo, siempre escapado:

plantilla.php — php-paso-a-paso/clase12 — Visual Studio Code

plantilla.php

```php
<p>Hola, <?= htmlspecialchars($nombre) ?></p>
```



En cuanto la misma página muestra el formulario **y** procesa los datos (lo veremos con calma en el punto 10), necesita saber en qué momento está: ¿acabamos de llegar o nos han enviado algo? Hay tres formas de averiguarlo, y conviene conocerlas todas.

Forma 1: con `$_SERVER["REQUEST_METHOD"]` (para POST)

Cuando **abrimos** una página escribiendo su dirección o pulsando un enlace, la petición es **GET**. Cuando pulsamos el botón de un formulario con `method="post"`, es **POST**. Así que con POST basta con preguntar por el método:

09-enviado-post.php — php-paso-a-paso/clase12 — Visual Studio Code

09-enviado-post.php

```php
<?php
declare(strict_types=1);

$enviado = $_SERVER["REQUEST_METHOD"] === "POST";
$nombre = trim($_POST["nombre"] ?? "");
?>
<?php if ($enviado): ?>
    <p>✅ Formulario recibido. Hola, <?= htmlspecialchars($nombre) ?>.</p>
<?php else: ?>
    <p>Todavía no has enviado el formulario.</p>
<?php endif; ?>

<!-- Sin action: se envía a esta misma página -->
<form method="post">
    <input type="text" name="nombre" placeholder="Nombre">
    <button type="submit">Enviar</button>
</form>
```

📄 Ejemplo: `09-enviado-post.php`

Forma 2: con `isset()` de un campo (imprescindible con GET)

Aquí está la trampa: con un formulario GET, `REQUEST_METHOD` **no sirve**. Al abrir la página, el método es `GET`... y al enviar el formulario, **también** es `GET`. Lo que cambia es que, al enviar, `$_GET` trae los campos. Así que preguntamos si existe uno:

10-enviado-get.php — php-paso-a-paso/clase12 — Visual Studio Code

10-enviado-get.php

```php
<?php
declare(strict_types=1);

// Al abrir la página, $_GET está vacío; al enviar, trae "nombre" (aunque esté en blanco)
$enviado = isset($_GET["nombre"]);
$nombre = trim($_GET["nombre"] ?? "");
?>
<p>Método de esta petición: <strong><?= $_SERVER["REQUEST_METHOD"] ?></strong></p>
<!-- ...el mismo mensaje y el mismo formulario, ahora con method="get" -->
```

El ejemplo enseña el método en pantalla: comprobad que dice `GET` las dos veces, aunque solo la segunda se haya enviado el formulario.

📄 Ejemplo: `10-enviado-get.php`

Forma 3: con el `name` del botón (sirve para los dos métodos)

Si el botón tiene `name`, el botón pulsado también viaja con el formulario. Su clave solo existe cuando alguien lo ha pulsado:

11-boton-post.php — php-paso-a-paso/clase12 — Visual Studio Code

11-boton-post.php

```php
<?php
declare(strict_types=1);

$enviado = isset($_POST["enviar"]);   // ¿se ha pulsado el botón "enviar"?
$nombre = trim($_POST["nombre"] ?? "");
?>
<?php if ($enviado): ?>
    <p>✅ Has pulsado «Enviar». Hola, <?= htmlspecialchars($nombre) ?>.</p>
<?php endif; ?>

<form method="post">
    <input type="text" name="nombre" placeholder="Nombre">
    <button type="submit" name="enviar">Enviar</button>
</form>
```

📄 Ejemplo: `11-boton-post.php`

Con GET es exactamente igual, cambiando `$_POST` por `$_GET`. Al enviar, veréis el botón en la URL: `12-boton-get.php?nombre=Ana&enviar=`.

📄 Ejemplo: `12-boton-get.php`

Varios botones en el mismo formulario

La gran ventaja del botón con `name` es que permite tener **varios** y saber cuál se ha pulsado, porque **solo viaja ese**. Podéis darles nombres distintos (`name="sumar"`, `name="restar"`) o, como aquí, el **mismo** `name` con distinto `value`:

13-varios-botones.php — php-paso-a-paso/clase12 — Visual Studio Code

13-varios-botones.php

```php
<?php
declare(strict_types=1);

$importe = (float) ($_POST["importe"] ?? 0);
$accion = $_POST["accion"] ?? "";
$resultado = "";

if ($accion === "iva") {
    $resultado = "Con IVA: " . number_format($importe * 1.21, 2, ",", ".") . " €";
} elseif ($accion === "descuento") {
    $resultado = "Con un 10 % de descuento: " . number_format($importe * 0.9, 2, ",", ".") .
" €";
}
?>
<form method="post">
    <input type="number" step="0.01" name="importe" value="<?= $importe ?>">
    <button type="submit" name="accion" value="iva">Añadir IVA</button>
    <button type="submit" name="accion" value="descuento">Aplicar descuento</button>
</form>
```

📄 Ejemplo: `13-varios-botones.php`

Resumen: ¿cuál uso?

Forma	Con POST	Con GET	Ideal para…
`$_SERVER["REQUEST_METHOD"] === "POST"`	✅	❌ Siempre es GET	Formularios POST con un solo botón
`isset($_GET["campo"])` / `isset($_POST["campo"])`	✅	✅	Formularios GET, como un buscador
`isset($_POST["boton"])` con `name` en el botón	✅	✅	Formularios con **varios** botones



💡 **¿Y si el usuario pulsa Intro en lugar del botón?** Los navegadores actuales hacen como si hubiera pulsado el **primer** botón del formulario, así que su `name` también viaja. Aun así, en un formulario POST con un solo botón, `REQUEST_METHOD` es la opción más robusta: no depende de ningún botón. Es la que usaremos en los ejemplos que siguen.



CLASE 12 · PUNTO 9

👍 Un caso real con dos páginas: inscribirse en un taller

Ya tenemos todas las piezas. Vamos a juntarlas en un caso real: la inscripción a un taller, que ahora sí **comprueba** los datos. Empezamos como en los puntos 2 y 3, con **dos archivos**: uno **pregunta** (el formulario) y otro **responde** (recibe los datos, los valida y contesta).

Esquema

```
14-inscripcion.html                          15-recibir-inscripcion.php
┌──────────────────────┐          POST      ┌──────────────────────────┐
│ Nombre: [ Ana      ] │ ─────────────────► │ $_POST["nombre"] → "Ana" │
│ Email:  [ ana@...  ] │                    │ $_POST["email"]  → "ana@…"│
│      [ Inscribirme ] │                    │ valida y responde        │
└──────────────────────┘                    └──────────────────────────┘
```

La página que **pregunta** es HTML puro. Lo importante son dos atributos del `<form>` (`14-inscripcion.html`):

plantilla.php — php-paso-a-paso/clase12 — Visual Studio Code

plantilla.php

```html
<!-- action: QUIÉN recibe los datos. method: CÓMO viajan -->
<form action="15-recibir-inscripcion.php" method="post">
    <p>
        <label for="nombre">Nombre</label>
        <input type="text" id="nombre" name="nombre">
    </p>
    <p>
        <label for="email">Email</label>
        <input type="text" id="email" name="email">
    </p>
    <button type="submit">Inscribirme</button>
</form>
```

- `action` es la dirección de la página que recibirá los datos.
- `method` dice cómo viajan: `get` (en la URL) o `post` (ocultos).
- El `name` de cada campo será la **clave** con la que lo leeremos en PHP.

Y la página que **responde** (`15-recibir-inscripcion.php`):

ejemplo.php — php-paso-a-paso/clase12 — Visual Studio Code

ejemplo.php

```php
<?php
declare(strict_types=1);

// Esta página SOLO recibe: los datos llegan en $_POST con la clave del name de cada campo
$errores = [];

$nombre = trim($_POST["nombre"] ?? "");
$email = trim($_POST["email"] ?? "");

if ($nombre === "") {
    $errores[] = "El nombre es obligatorio.";
}
if (filter_var($email, FILTER_VALIDATE_EMAIL) === false) {
    $errores[] = "El email no tiene un formato válido.";
}
?>
<?php if (count($errores) === 0): ?>
    <h1>¡Inscripción recibida!</h1>
    <p>Gracias, <?= htmlspecialchars($nombre) ?>. Te escribiremos a <?=
htmlspecialchars($email) ?>.</p>
<?php else: ?>
    <h1>Faltan datos</h1>
    <ul>
        <?php foreach ($errores as $error): ?>
            <li><?= $error ?></li>
        <?php endforeach; ?>
    </ul>
    <!— El usuario tiene que volver atrás… y puede que lo escrito se haya perdido —>
    <p><a href="14-inscripcion.html">← Volver al formulario</a></p>
<?php endif; ?>
```

Abrid `14-inscripcion.html`, rellenadlo y enviadlo. Fijaos en la barra de direcciones: **habéis cambiado de página**, de `14-inscripcion.html` a `15-recibir-inscripcion.php`.

Funciona, y es muy fácil de entender: cada archivo hace **una sola cosa**. Pero haced estas tres pruebas:

Prueba	Qué pasa	¿Problema?
Enviad el formulario vacío	Sale la lista de errores en **otra** página	El usuario tiene que pulsar «Volver» y buscar qué estaba mal
Escribid el nombre con un email mal escrito y volved	Según el navegador, el nombre sigue escrito… o no	No podemos rellenar los campos desde PHP: el formulario es HTML puro
Abrid `15-recibir-inscripcion.php` directamente, sin pasar por el formulario	Salen los errores aunque nadie ha enviado nada	La página que recibe no sabe si la han llamado desde el formulario



CLASE 12 · PUNTO 10

🚀 Todo en la misma página

La solución a esos tres problemas es que **la misma página** pregunte **y** responda. El formulario se envía a sí mismo: basta con **no poner** `action` (o dejarlo vacío). Pero entonces la página necesita saber en cuál de las dos situaciones está:

- Si la hemos **abierto** escribiendo la dirección o con un enlace, llega por **GET**: solo hay que enseñar el formulario.
- Si venimos de **pulsar el botón**, llega por **POST**: hay que validar y responder.

Eso nos lo dice `$_SERVER["REQUEST_METHOD"]` (`16-una-sola-pagina.php`):

ejemplo.php — php-paso-a-paso/clase12 — Visual Studio Code

ejemplo.php

```php
<?php
declare(strict_types=1);

$errores = [];
$inscrito = false;

// ¿Venimos de pulsar el botón (POST) o solo estamos abriendo la página (GET)?
$esEnvio = $_SERVER["REQUEST_METHOD"] === "POST";
$nombre = trim($_POST["nombre"] ?? "");
$email = trim($_POST["email"] ?? "");

if ($esEnvio) {
    if ($nombre === "") {
        $errores[] = "El nombre es obligatorio.";
    }
    if (filter_var($email, FILTER_VALIDATE_EMAIL) === false) {
        $errores[] = "El email no tiene un formato válido.";
    }
    $inscrito = count($errores) === 0;
}
?>
<?php if ($inscrito): ?>
    <p>Gracias, <?= htmlspecialchars($nombre) ?>. Te escribiremos a <?=
htmlspecialchars($email) ?>.</p>
<?php else: ?>
    <?php foreach ($errores as $error): ?>
        <p style="color: #b00020"><?= $error ?></p>
    <?php endforeach; ?>

    <!-- Sin action: el formulario se envía a esta misma página -->
    <form method="post">
        <input type="text" id="nombre" name="nombre" value="<?= htmlspecialchars($nombre) ?
>">
        <input type="text" id="email" name="email" value="<?= htmlspecialchars($email) ?>">
        <button type="submit">Inscribirme</button>
    </form>
<?php endif; ?>
```

Repetid las tres pruebas de antes:

- Los errores salen **encima del formulario**, sin cambiar de página.
- Lo que el usuario escribió **sigue en los campos**, porque ahora el formulario es PHP y lo rellenamos con `value="..."`.
- Si se abre la página sin enviar nada, `$esEnvio` es `false` y no sale ningún error: solo el formulario vacío.



CLASE 12 · PUNTO 11

🤔 ¿Una página o dos? Lo que gana y lo que pierde cada una

	Dos páginas	Una sola página
Errores	En otra página; el usuario tiene que volver	Junto al formulario, en el sitio
Datos escritos	Se pueden perder al volver	Se conservan siempre (`value`)
Abrir la página «a pelo»	La que recibe muestra errores falsos	Distingue GET de POST
Archivos que mantener	Dos, y sus `name` tienen que coincidir	Uno
Lo fácil de leer	Cada archivo hace **una** cosa	El archivo hace **dos**: decidir y pintar

Para el **usuario**, gana claramente la página única, y por eso es la que usaremos a partir de ahora. Pero fijaos en la última fila: hemos juntado en un solo archivo dos trabajos muy distintos:

1. **Decidir**: leer `$_POST`, validar, guardar, elegir qué mensaje se muestra. Es **lógica**.
2. **Pintar**: el HTML, con sus etiquetas y sus estilos. Es **presentación**.

Con un formulario de dos campos no pasa nada. Pero con veinte campos, una consulta a la base de datos y una página con mucho diseño, ese archivo se convierte en una sopa de PHP y HTML donde nadie encuentra nada. Y si en vuestro equipo hay alguien que solo sabe de diseño, tendrá que pelearse con vuestro código para cambiar un color. 😅



🎯 **¿Entonces qué es mejor?** Ni una cosa ni la otra tal cual: **una sola dirección para el usuario, pero el trabajo repartido en dos archivos para nosotros**. Lo vemos ahora mismo.



CLASE 12 · PUNTO 12

😮 Separar la lógica de la vista: la semilla del MVC

Vamos a coger la página única y partirla en dos archivos, cada uno con **un solo trabajo** (`17-separado/`):

Esquema

```
17-separado/
├── inscripcion.php              ← el que DECIDE: lee, valida y elige qué mostrar
└── vistas/
    └── inscripcion.vista.php    ← el que PINTA: solo HTML con huecos para los datos
```

El primero, el que decide, no escribe **ni una etiqueta HTML** (`inscripcion.php`):

ejemplo.php — php-paso-a-paso/clase12 — Visual Studio Code

ejemplo.php

```php
<?php
declare(strict_types=1);

// EL CONTROLADOR: recibe la petición, comprueba los datos y decide qué se muestra.
// No escribe ni una etiqueta HTML.

function validarInscripcion(string $nombre, string $email): array
{
    $errores = [];
    if ($nombre === "") {
        $errores[] = "El nombre es obligatorio.";
    }
    if (filter_var($email, FILTER_VALIDATE_EMAIL) === false) {
        $errores[] = "El email no tiene un formato válido.";
    }
    return $errores;
}

$errores = [];
$inscrito = false;

$esEnvio = $_SERVER["REQUEST_METHOD"] === "POST";
$nombre = trim($_POST["nombre"] ?? "");
$email = trim($_POST["email"] ?? "");

if ($esEnvio) {
    $errores = validarInscripcion($nombre, $email);
    $inscrito = count($errores) === 0;
    // Aquí iría guardar la inscripción en un fichero o en la base de datos
}

// LA VISTA: solo pinta con los datos que le hemos preparado
require __DIR__ . "/vistas/inscripcion.vista.php";
```

Y el segundo, la **vista**, es el HTML de antes, sin una sola decisión «de negocio»: no lee `$_POST` , no valida, no guarda. Solo usa las variables que le ha dejado preparadas el primero: `$inscrito` , `$errores` , `$nombre` y `$email` (`inscripcion.vista.php`).

Para el usuario no ha cambiado nada: sigue habiendo **una sola dirección**, `inscripcion.php` , que pregunta y responde. Pero para nosotros todo está en su sitio:

¿Quiero cambiar...?	Abro...
Una regla de validación	`inscripcion.php`
El diseño, un texto o un color	`vistas/inscripcion.vista.php`
Dónde se guardan los datos	`inscripcion.php` (y, más adelante, otro archivo solo para eso)

Esto tiene nombre: MVC

Lo que acabáis de hacer es la idea en la que se basan **todos** los *frameworks* modernos, Laravel incluido. Se llama **MVC** (*Modelo, Vista, Controlador*) y la construiremos a lo grande a partir de la clase 23:

Esquema

```
petición ──►  ┌──────────────────────┐   lee los datos, valida y decide
              │      CONTROLADOR     │
              │   inscripcion.php    │
              └──────┬────────┬──────┘
                     │        │
          pide datos ▼        ▼ le pasa los datos
        ┌──────────────┐  ┌──────────────────────┐
        │    MODELO    │  │        VISTA         │
        │ (fichero, BD)│  │ inscripcion.vista.php│ ──► HTML
        └──────────────┘  └──────────────────────┘
```

- El **controlador** es el camarero y jefe de sala del restaurante: recibe la comanda, la comprueba y organiza.
- La **vista** es el emplatado: cómo se presenta el plato en la mesa.
- El **modelo**, que todavía no tenemos, será la cocina y la despensa: guardar y leer los datos (en la clase 15, de la base de datos).



💡 No os agobiéis con los nombres: de momento basta con una costumbre. **Arriba, la lógica; abajo, la vista**, y cuando la página crezca, la vista a su propio archivo. Es exactamente el orden de programa que seguimos desde la clase 10: entradas y procesamiento primero, salidas al final.



CLASE 12 · PUNTO 13

👌 El patrón del formulario profesional

Ahora que sabéis por qué, vamos con un formulario de contacto completo que se procesa **en el mismo archivo**, con la lógica arriba y la vista abajo. Fijaos en que sigue el orden de programa que vimos en la clase 10, porque lo usaréis siempre:

Parte	En un formulario
Funciones	`e()` para escapar y `validarContacto()`, que devuelve un array con los errores
Inicialización	`$errores = [];` y `$enviado = false;`
Entradas	¿Me han enviado el formulario? (`$_SERVER["REQUEST_METHOD"]`) y los datos, limpios con `trim()`
Procesamiento	Si es un envío: validar y, si no hay errores, procesar (guardar, enviar...)
Salidas	El HTML: los errores y el formulario con lo que el usuario ya había escrito

Y en código:

18-formulario-contacto.php — php-paso-a-paso/clase12 — Visual Studio Code

18-formulario-contacto.php

```php
<?php
declare(strict_types=1);

function e(string $texto): string
{
    return htmlspecialchars($texto, ENT_QUOTES, "UTF-8");
}

function validarContacto(string $nombre, string $email): array
{
    $errores = [];
    if ($nombre === "") {
        $errores[] = "El nombre es obligatorio.";
    }
    if (filter_var($email, FILTER_VALIDATE_EMAIL) === false) {
        $errores[] = "El email no tiene un formato válido.";
    }
    return $errores;
}

$errores = [];
$enviado = false;

$esEnvio = $_SERVER["REQUEST_METHOD"] === "POST";
$nombre = trim($_POST["nombre"] ?? "");
$email = trim($_POST["email"] ?? "");

if ($esEnvio) {
    $errores = validarContacto($nombre, $email);
    $enviado = count($errores) === 0;
}
?>
<!— y aquí, el HTML —>
```

En el HTML volvemos a poner en cada campo lo que el usuario escribió, para que no tenga que rellenarlo todo otra vez por un solo error:

18-formulario-contacto.php — php-paso-a-paso/clase12 — Visual Studio Code

18-formulario-contacto.php

```php
<input type="text" name="nombre" value="<?= e($nombre) ?>">
```

¿Y qué hace exactamente esa `e()` ? Lo explico en el punto 15.

📄 Ejemplo completo: `18-formulario-contacto.php`



💡 **¿Por qué `REQUEST_METHOD` y no `isset($_POST["enviar"])` ?** Las dos funcionan, como vimos en el punto 8. Pero este formulario tiene un solo botón y va por POST, así que preguntar por el método es lo más sencillo y no depende de que el botón tenga `name` . Si tuviera varios botones (guardar, borrar...), entonces sí miraríamos cuál se ha pulsado.

✅ Tres pruebas para comprobarlo

Haced esto	Debéis obtener
Enviad el formulario vacío	Tres errores en rojo
Escribid un nombre y un email mal escrito	Un error del email... **y el nombre sigue escrito**
Rellenadlo bien	El mensaje de agradecimiento



CLASE 12 · PUNTO 14

🤔 Validar: comprobar que el dato es correcto

Validar es comprobar que lo que llega tiene sentido **antes** de usarlo. Y ojo: el atributo `required` del HTML ayuda al usuario, pero **no os protege**. Cualquiera puede saltárselo desde las herramientas del navegador. La validación de verdad se hace **siempre en PHP**.

Qué comprobar	Cómo
Campo obligatorio	`trim($dato) === ""`
Longitud mínima o máxima	`mb_strlen($dato) < 10`
Email	`filter_var($email, FILTER_VALIDATE_EMAIL)` [2]
Número entero en un rango	`filter_var($edad, FILTER_VALIDATE_INT, ["options" => ["min_range" => 16, "max_range" => 99]])`
Valor de una lista (un `<select>`)	`in_array($ciudad, $ciudades, true)`
Dos campos iguales (contraseñas)	`$clave === $clave2`



🎯 **Truco:** si guardáis los errores con el nombre del campo como clave (`$errores["email"] = "..."`), luego podéis mostrar cada error **justo debajo de su campo**. Lo tenéis en la solución del ejercicio 01.



CLASE 12 · PUNTO 15

🐞 Escapar: htmlspecialchars y los ataques XSS

Imaginad un libro de visitas. Alguien escribe esto como «mensaje»:

19-xss.php — php-paso-a-paso/clase12 — Visual Studio Code

19-xss.php

```
1 <script>alert('¡Te he hackeado!')</script>
```

Si lo mostráis tal cual con `echo`, el navegador de **cada persona que visite la página** ejecutará ese código. Una alerta es inofensiva, pero en su lugar podría haber un script que robe sesiones. Este ataque se llama **XSS** (*Cross-Site Scripting*) y es de los más comunes en la web [3].

La defensa es sencilla: **escapar** todo lo que venga del usuario antes de mostrarlo. `htmlspecialchars()` convierte los caracteres peligrosos en texto inofensivo [4]:

Carácter	Se convierte en
`<`	`<`
`>`	`>`
`"`	`"`
`'`	`'`

El navegador **muestra** `<script>` en pantalla como texto, pero ya **no lo ejecuta**.

Como lo vais a escribir cientos de veces, mi recomendación es crear una función corta:

19-xss.php — php-paso-a-paso/clase12 — Visual Studio Code

19-xss.php

```php
<?php
function e(string $texto): string
{
    return htmlspecialchars($texto, ENT_QUOTES, "UTF-8");
}

echo "<p>" . e($comentario) . "</p>";
```

📄 Ejemplo: `19-xss.php`



🛡️ **Las dos reglas de oro: validad al entrar** (¿el dato tiene sentido?) y **escapad al salir** (¿se puede mostrar sin peligro?). Son dos cosas distintas, y hacen falta las dos.



CLASE 12 · PUNTO 16

👍 Guardar datos en un fichero JSON

Hasta que veamos bases de datos, podemos guardar información en **ficheros**. El formato más cómodo es **JSON**: un texto que representa arrays y que entienden PHP, JavaScript y casi cualquier lenguaje [5].

datos.json — php-paso-a-paso/clase12 — Visual Studio Code

datos.json

```json
[
    { "nombre": "Ana", "texto": "Hola clase", "fecha": "06/11/2026 17:45" }
]
```

Solo necesitáis cuatro funciones:

Función	Qué hace
`json_encode($array)`	Array → texto JSON
`json_decode($texto, true)`	Texto JSON → array (el `true` es para que devuelva un array asociativo)
`file_put_contents($ruta, $texto)`	Escribe un texto en un fichero (lo crea si no existe)
`file_get_contents($ruta)`	Lee un fichero entero y lo devuelve como texto

20-libro-visitas.php — php-paso-a-paso/clase12 — Visual Studio Code

20-libro-visitas.php

```php
<?php
$archivo = __DIR__ . "/datos/visitas.json";

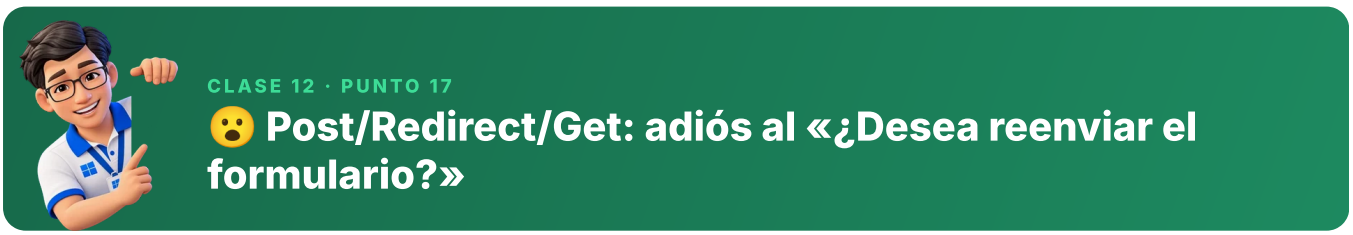
// Leer
$visitas = [];
if (file_exists($archivo)) {
    $visitas = json_decode(file_get_contents($archivo), true) ?? [];
}

// Añadir y guardar
$visitas[] = ["nombre" => $nombre, "texto" => $texto];
file_put_contents($archivo, json_encode($visitas, JSON_PRETTY_PRINT |
JSON_UNESCAPED_UNICODE), LOCK_EX);
```

Tres detalles de ese código:

- `__DIR__` es la carpeta donde está el archivo PHP. Con `__DIR__ . "/datos/..."` la ruta funciona aunque mováis el proyecto de sitio.
- `JSON_PRETTY_PRINT` y `JSON_UNESCAPED_UNICODE` hacen que el fichero sea legible y que las tildes se vean bien.
- `LOCK_EX` evita que dos personas escriban a la vez y se estropee el fichero.

📄 Ejemplo: `20-libro-visitas.php` . Abrid `datos/visitas.json` después de firmar: es texto normal.



Firmad en el libro de visitas y pulsad **F5**. Si no hacemos nada, el navegador pregunta «¿Desea reenviar el formulario?»... y si decís que sí, la firma se guarda **dos veces**. En una tienda, sería un pedido duplicado. 😬

La solución es un patrón con nombre propio: **Post/Redirect/Get**. Después de procesar el POST con éxito, **redirigimos** a la misma página con una petición GET:

ejemplo.php — php-paso-a-paso/clase12 — Visual Studio Code

ejemplo.php

```php
<?php
// ...guardar los datos...
header("Location: 20-libro-visitas.php?ok=1");
exit;
```

Ahora F5 recarga el GET, que es inofensivo. Y el `?ok=1` nos sirve para mostrar el mensaje de «¡Gracias por firmar!».



⚠️ **Nunca olvidéis el `exit;` después de `header("Location: ...")`**. La cabecera le dice al navegador que se vaya, pero PHP **sigue ejecutando** lo que hay debajo si no lo paráis [6].

⚠️ Y `header()` tiene que ir **antes de cualquier salida**: ni un `echo`, ni HTML, ni un espacio en blanco antes. Si no, veréis el famoso error *«headers already sent»*. Por eso procesamos el formulario arriba del todo y el HTML va después.



CLASE 12 · PUNTO 18

🚀 Subir archivos

Para subir fotos o documentos, el formulario necesita dos cosas: `method="post"` y, **obligatoriamente**, `enctype="multipart/form-data"`. Sin ese atributo, el archivo no viaja.

21-subir-foto.php — php-paso-a-paso/clase12 — Visual Studio Code

21-subir-foto.php

```html
<form method="post" enctype="multipart/form-data">
    <input type="file" name="foto">
    <button type="submit">Subir</button>
</form>
```

Los archivos no llegan a `$_POST`, sino a `$_FILES` [7]:

Clave en `$_FILES["foto"]`	Qué contiene
`["name"]`	El nombre original en el ordenador del usuario. **No lo uséis para guardar**
`["tmp_name"]`	Dónde lo ha dejado PHP temporalmente. Es el archivo de verdad
`["size"]`	Tamaño en bytes
`["error"]`	`UPLOAD_ERR_OK` (vale 0) si todo ha ido bien

Las comprobaciones que no podéis saltaros

21-subir-foto.php — php-paso-a-paso/clase12 — Visual Studio Code

21-subir-foto.php

```php
<?php
$foto = $_FILES["foto"] ?? null;
$tiposPermitidos = ["image/jpeg" => "jpg", "image/png" => "png", "image/webp" => "webp"];

if ($foto === null || $foto["error"] !== UPLOAD_ERR_OK) {
    $mensaje = "No ha llegado ningún archivo.";
} elseif ($foto["size"] > 2 * 1024 * 1024) {
    $mensaje = "Máximo 2 MB.";
} else {
    $info = getimagesize($foto["tmp_name"]);     // ¿es de verdad una imagen?
    $tipo = $info["mime"] ?? "";

    if (!isset($tiposPermitidos[$tipo])) {
        $mensaje = "Solo JPG, PNG o WEBP.";
    } else {
        $nombreFinal = bin2hex(random_bytes(8)) . "." . $tiposPermitidos[$tipo];
        move_uploaded_file($foto["tmp_name"], __DIR__ . "/uploads/" . $nombreFinal);
    }
}
```

¿Por qué tanto lío?

- **Comprobamos el contenido, no el nombre.** Cualquiera puede llamar `foto.jpg` a un programa malicioso. `getimagesize()` mira dentro del archivo.
- **Inventamos un nombre nuevo y aleatorio.** Así nadie puede sobrescribir otro archivo ni colar un nombre peligroso.
- `move_uploaded_file()` mueve el archivo temporal a su sitio definitivo. Si no lo movéis, PHP lo borra al terminar.

📄 Ejemplo: `21-subir-foto.php`



💡 Si una foto grande da error aunque esté por debajo de vuestro límite, revisad `upload_max_filesize` y `post_max_size` en el `php.ini` (en Laragon: **Menú → PHP → php.ini**). Recordad que con `phpinfo()` podéis ver qué valores tenéis.



CLASE 12 · PUNTO 19

🔍 Para saber más: ficheros CSV

Un **CSV** es una tabla en texto: una fila por línea y los valores separados por `;` o `,`. Excel los abre directamente, así que son perfectos para exportar listados.

22-csv.php — php-paso-a-paso/clase12 — Visual Studio Code

22-csv.php

```php
<?php
$f = fopen(__DIR__ . "/datos/alumnos.csv", "w");    // abrir para escribir
fputcsv($f, ["Marta", 22, "Santander"], ";");
fclose($f);                                          // cerrar siempre
```

Para leerlos se usa `fgetcsv()` dentro de un `while`, línea a línea.

📄 Ejemplo: `22-csv.php`



CLASE 12 · PUNTO 20

🚀 Ejercicios

- ☐ **01 · Registro.** Nombre, email, edad (entre 16 y 99), ciudad (un `<select>`) y contraseña repetida dos veces (mínimo 8 caracteres). Cada error debe aparecer **debajo de su campo** y los datos deben conservarse (menos las contraseñas). → `ej01-registro.php`
- ☐ **02 · Encuesta.** «¿Qué lenguaje te gusta más?» con botones de radio. Guardad los votos en un JSON, aplicad Post/Redirect/Get y mostrad los resultados con su porcentaje. → `ej02-encuesta.php`
- ☐ **03 · Galería.** Un formulario para subir fotos a la carpeta `fotos/` y, debajo, todas las fotos subidas. Pista: `glob()` os da la lista de archivos de una carpeta. → `ej03-galeria.php`
- ☐ **04 · Calculadora web.** Dos números y una operación en un `<select>`. Validad que son números (`is_numeric()`) y la división entre cero, conservad lo escrito y mostrad el resultado. → `ej04-calculadora.php`
- ☐ **05 · Conversor de unidades.** Un formulario **GET** que convierta kilómetros y millas, kilos y libras. Comprobad que el resultado se puede compartir copiando la dirección. → `ej05-conversor.php`

- ☐ **06 · Lista de tareas.** Añadir, marcar como hecha y borrar tareas guardadas en un JSON, siempre con Post/Redirect/Get. Pista: un botón puede llevar `name` y `value`, y así un mismo formulario sabe qué botón se pulsó. → `ej06-tareas.php`
- ☐ **07 · Contador de visitas.** Guardad en `datos/visitas.txt` cuántas veces se ha abierto la página y mostradlo. → `ej07-contador-visitas.php`
- ☐ **08 · Envía tu CV.** Subida de un PDF de 2 MB como máximo, comprobando el tipo real con `mime_content_type()` y guardándolo en `cv/` con un nombre nuevo. → `ej08-subir-cv.php`
- ☐ **09 · Descargar en CSV.** Una página que, al abrirla, **descargue** un `alumnos.csv` que se abra bien en Excel. Pistas: las cabeceras `Content-Type` y `Content-Disposition`, y `fopen("php://output", "w")`. → `ej09-exportar-csv.php`
- ☐ **10 · Leer un CSV.** Leed `datos/productos.csv` y mostradlo en una tabla con el importe de cada línea y el total. Cuidado: los precios vienen con coma decimal. → `ej10-importar-csv.php`
- ☐ **11 · Hamburguesa a medida.** Casillas de extras con `name="extras[]"` (llega un array) y el precio total. Ignorad cualquier extra que no exista en vuestra lista. → `ej11-pedido-extras.php`



🏆 **Reto:** en el libro de visitas, probad a firmar con `<b>Hola</b>` como mensaje. ¿Sale en negrita? ¿Por qué no? Ahora quitad la `e()` en esa línea y repetid la prueba. Volved a ponerla en cuanto lo veáis. 😉



CLASE 12 · PUNTO 21

👋 Lo que tenéis que llevaros de hoy

Idea	En una frase
Las piezas	`action` (adónde), `method` (cómo) y un `name` en cada campo (la clave en PHP)
GET o POST	GET para consultar, POST para enviar o cambiar datos. Contraseñas, siempre por POST
Todo llega como texto	`(int)` o `(float)` antes de operar; las casillas y los radios sin marcar no llegan
Leer sin sustos	`trim($_POST["campo"] ?? "")` y, para mirar qué llega, `var_dump($_POST)`
Dos páginas	`action="recibir.php"`: fácil de entender, pero los errores salen lejos y se pierde lo escrito
Una página	Sin `action`: el formulario se envía a sí mismo y conserva los datos
¿Me han enviado el formulario?	POST: `$_SERVER["REQUEST_METHOD"] === "POST"`. GET: `isset($_GET["campo"])`. Los dos: un `name` en el botón
Lógica y vista	Una dirección para el usuario, dos archivos para nosotros: la semilla del MVC
Validar	En PHP, siempre. El `required` del HTML no protege
Escapar	`htmlspecialchars()` (nuestra `e()`) en todo lo que venga del usuario
Conservar datos	`value="<?= e($nombre) ?>"`
JSON	`json_encode` / `json_decode` + `file_put_contents` / `file_get_contents`
PRG	Tras un POST correcto: `header("Location: ...")` y `exit;`
Subidas	`enctype="multipart/form-data"`, `$_FILES`, comprobar tipo y tamaño, nombre aleatorio

📚 Fuentes y documentación oficial

1. Manual de PHP: superglobales
2. Manual de PHP: filtros de validación
3. OWASP: Cross Site Scripting (XSS)
4. Manual de PHP: htmlspecialchars
5. Manual de PHP: json_encode
6. Manual de PHP: header
7. Manual de PHP: subida de ficheros con el método POST
8. MDN: enviar los datos de un formulario
9. Manual de PHP: variables desde fuentes externas